

Comprendre mon script JavaScript

Étude complète du portfolio de Jonathan Fradet

Cours pédagogique basé sur les fichiers **script.js**, **index.html** et **assets/css/style.css**.
Analyse de la version du projet consultée le 20 juillet 2026.

Objectif : comprendre les données, l'ordre d'exécution, le DOM, les animations et chaque interaction du portfolio.

Table des matières

Les numéros de page sont calculés automatiquement à partir de la mise en page finale.

1. Présentation générale du script	5
1.1 Les grandes parties du programme	5
1.2 Résumé très simple	5
2. Contexte et ordre réel d'exécution	6
2.1 Le petit script du head	6
2.2 Exécution de script.js de haut en bas	6
2.3 Schéma du chargement initial	7
3. Comprendre les objets de contenu	8
3.1 Clé, propriété et valeur	8
3.2 Pourquoi la même structure ?	8
3.3 Le booléen afficherProjets	8
3.4 Ce que const garantit réellement	9
4. Fonctions, paramètres et arguments	10
4.1 Un paramètre peut représenter plusieurs objets	10
4.2 Booléens et variable locale	10
4.3 Cas limite révélateur	10
5. Explication complète des conditions	11
5.1 Le point d'exclamation	11
5.2 Démonstration avec les trois objets	11
5.3 Pourquoi deux if séparés ?	12
6. Le DOM et les éléments HTML	13
6.1 Ce que renvoie getElementById	13
6.2 Les éléments sont initialement vides	13
6.3 Le risque de null	13
7. Différence entre fonction, méthode et constructeur	14
7.1 Décomposer element.classList.add	14
7.2 Le cas de setTimeout	14
8. Fonction interne, portée et closure	15
8.1 Pourquoi la fonction interne voit les variables	15
8.2 Pourquoi ne pas passer quatre arguments ?	15
8.3 Une closure par appel	15
9. Mise à jour du contenu	16

9.1 innerHTML et contenu.texte	16
9.2 src et alt	16
9.3 Flux complet des données	16
10. Tableaux, forEach et callbacks	17
10.1 Déroulement de forEach	17
10.2 Fonction fléchée et callback	17
10.3 Pourquoi éviter la répétition	17
10.4 Le tableau des sources	18
11. Classes CSS et classList	19
11.1 Les fondus du contenu principal	19
11.2 Le bloc Projets	19
11.3 Pourquoi retirer visible trop tôt casse l'effet	20
12. Explication approfondie de setTimeout	21
12.1 Pourquoi le programme n'est pas bloqué	21
12.2 Premier délai : masquer le bloc Projets	21
12.3 Deuxième délai : remplacer le contenu	22
12.4 Troisième délai : fermer la navigation	22
13. Préchargement des images	23
13.1 Les trois itérations	23
13.2 Cache, réseau et limite du code	23
14. Le premier affichage	24
14.1 Pourquoi aucun fade-out ?	24
15. Gestion du menu burger	25
15.1 Ouvrir et fermer avec toggle	25
15.2 Fermer après un clic dans la navigation	25
15.3 Chronologie du menu	26
15.4 Accessibilité actuelle	26
16. Chronologies exactes des interactions	27
16.1 Clic sur Projets	27
16.2 Clic sur Accueil	27
16.3 Clic sur Compétences	27
16.4 Quitter Projets	28
16.5 Clics rapides et callbacks concurrents	28
17. Tableau récapitulatif des notions	29
18. Lecture complète de script.js, bloc par bloc	31
18.1 Déclaration des trois objets	31

18.2 Création de sourcesImages	31
18.3 Déclaration et appel du préchargement	31
18.4 Entrée de changerContenu et sélection du DOM	32
18.5 Fonction interne de mise à jour	32
18.6 Branche du premier affichage	32
18.7 Début de la branche animée	33
18.8 Masquage conditionnel des projets	33
18.9 Remplacement différé et entrée	33
18.10 Initialisation et menu	33
19. Schémas de fonctionnement	35
19.1 Chargement initial	35
19.2 Mise à jour du contenu	35
19.3 Clic sur Projets	35
19.4 Clic sur Accueil après Projets	36
19.5 Clic sur Compétences	36
19.6 Préchargement et menu	36
20. Subtilités, erreurs possibles et diagnostic	37
21. Pistes d'amélioration futures	40
22. Exercices de compréhension	41
23. Corrections expliquées	43
24. Glossaire final	45

1. Présentation générale du script

Le fichier `script.js` transforme une page HTML principalement vide en une interface de portfolio interactive. Il ne recharge pas la page lorsque l'utilisateur choisit Accueil, Compétences ou Projets. Il remplace plutôt le titre, le texte et l'image déjà présents dans le DOM, puis il ajoute ou retire des classes CSS pour déclencher les animations.

À retenir

Le JavaScript décide quelles données afficher et à quel moment. Le HTML fournit les éléments à modifier. Le CSS décide de leur apparence, de leur disposition et de leurs animations.

1.1 Les grandes parties du programme

- Trois objets de contenu regroupent les données d'Accueil, de Compétences et de Projets.
- Un tableau regroupe les trois chemins d'images à précharger.
- La fonction `prechargerImages` lance le chargement anticipé des images.
- La fonction `changerContenu` orchestre chaque changement d'écran.
- La fonction interne `mettreAJourContenu` affecte les nouvelles valeurs aux éléments HTML.
- Deux conditions décident s'il faut masquer ou afficher le bloc des liens de projets.
- Des temporisations de 300 ms synchronisent le remplacement avec les fondus de sortie.
- Deux écouteurs d'événements gèrent l'ouverture et la fermeture du menu responsive.

```
Données dans les objets
  |
  v
changerContenu(contenu, premierAffichage)
  |
  +--> modifie le DOM
  |
  +--> ajoute ou retire des classes
        |
        v
        le CSS anime la page
```

1.2 Résumé très simple

Au démarrage, le script prépare les images puis affiche Accueil. Lors d'un clic, il fait disparaître l'ancien contenu, attend environ 300 ms, insère les nouvelles données et relance un fondu entrant. Si l'objet choisi est `projets`, il affiche aussi le bloc contenant les cartes de projets.

Attention

Le bloc Projets ne fait pas partie du tableau des trois éléments animés. Il possède ses propres classes, ses propres règles CSS et une durée d'entrée différente.

2. Contexte et ordre réel d'exécution

Un navigateur lit normalement le document HTML de haut en bas. Dans ce projet, le fichier principal `script.js` est chargé juste avant la fermeture de `body`. Les éléments qu'il recherche ont donc déjà été créés dans le DOM.

```
<script src="script.js"></script>

</body>
</html>
```

Pourquoi cet emplacement compte

Si le script était exécuté avant la création du titre, du paragraphe, de l'image ou du menu, `getElementById` pourrait renvoyer null. Ici, l'emplacement en bas de page évite ce problème.

2.1 Le petit script du head

Le fichier `index.html` contient aussi un petit script indépendant dans le `head`. Son seul rôle est de retirer la classe `css-loading` lorsque la feuille de style est disponible, en erreur, ou après un délai de secours de 3000 ms. Le présent cours ne le confond pas avec `script.js`, mais il faut le mentionner pour décrire le chargement réel de la page.

2.2 Exécution de script.js de haut en bas

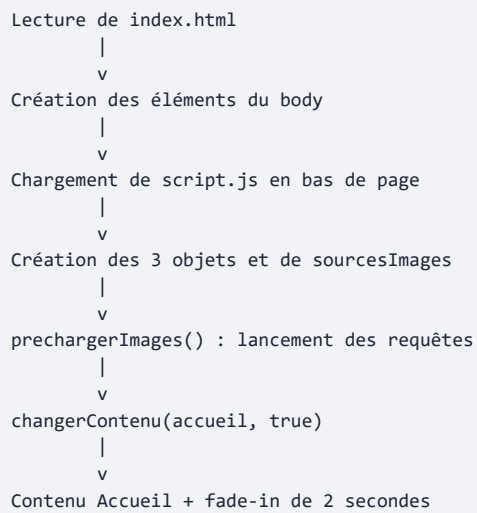
Avant d'exécuter les instructions visibles, le moteur prépare l'environnement global : les noms des fonctions déclarées avec `function` sont déjà disponibles grâce au hoisting. Il exécute ensuite les initialisations et les appels dans l'ordre du fichier. Le corps d'une fonction ne s'exécute toujours qu'au moment d'un appel.

- | | |
|---|--|
| 1 | Création de la constante <code>accueil</code> et de son objet. |
| 2 | Création de <code>competences</code> . |
| 3 | Création de <code>projets</code> . |
| 4 | Création du tableau <code>sourcesImages</code> avec trois chaînes de caractères. |
| 5 | Le flux rencontre les déclarations de <code>prechargerImages</code> et <code>changerContenu</code> sans exécuter leur corps. |
| 6 | Exécution de <code>prechargerImages()</code> . |
| 7 | Exécution immédiate de <code>changerContenu(accueil, true)</code> . |
| 8 | Recherche du bouton burger et de la navigation. |
| 9 | Installation des deux écouteurs de clic. |

Déclaration et appel

Écrire `function changerContenu(...)` déclare la fonction, mais n'exécute pas son corps. Le corps commence seulement lorsqu'un appel avec des parenthèses est rencontré.

2.3 Schéma du chargement initial



```
graph TD; A[Lecture de index.html] --> B[Création des éléments du body]; B --> C[Chargement de script.js en bas de page]; C --> D[Création des 3 objets et de sourcesImages]; D --> E[prechargerImages() : lancement des requêtes]; E --> F[changerContenu(accueil, true)]; F --> G[Contenu Accueil + fade-in de 2 secondes];
```

Lecture de index.html
|
v
Création des éléments du body
|
v
Chargement de script.js en bas de page
|
v
Création des 3 objets et de sourcesImages
|
v
prechargerImages() : lancement des requêtes
|
v
changerContenu(accueil, true)
|
v
Contenu Accueil + fade-in de 2 secondes

3. Comprendre les objets de contenu

Un objet JavaScript regroupe plusieurs informations liées. Les accolades délimitent l'objet. Chaque ligne associe une clé, placée avant les deux-points, à une valeur placée après.

```
const accueil = {  
  titre: "Bienvenue sur mon portfolio",  
  texte: `...`,  
  image: 'images/villedenuit1.jpg',  
  afficherProjets: false  
};
```

Objet	titre	image	afficherProjets
accueil	Bienvenue sur mon portfolio	images/villedenuit1.jpg	false
competences	Mes compétences	images/competences-v2.png	false
projets	Mes projets	images/liberté.jpg	true

3.1 Clé, propriété et valeur

Dans `titre: "Mes projets"`, `titre` est la clé ou propriété et `"Mes projets"` est sa valeur. La notation avec un point lit une propriété :

```
accueil.image // "images/villedenuit1.jpg"  
projets.afficherProjets // true
```

- `titre` fournit le contenu du `h2`.
- `texte` fournit le contenu du paragraphe et contient des balises `br`.
- `image` fournit le chemin affecté à l'attribut `src`.
- `afficherProjets` sert d'indicateur logique pour le bloc de cartes.

3.2 Pourquoi la même structure ?

Les trois objets possèdent les mêmes propriétés afin qu'une seule fonction puisse traiter les trois sections. La logique ne cherche pas à savoir si elle a reçu Accueil ou Compétences par son nom. Elle lit simplement `contenu.titre`, `contenu.texte`, `contenu.image` et `contenu.afficherProjets`.

Séparation utile

Les objets contiennent les données. La fonction contient la logique. Les éléments HTML reçoivent l'affichage. Cette organisation évite trois fonctions presque identiques.

3.3 Le booléen afficherProjets

Appel	contenu représente	contenu.afficherProjets	Décision
<code>changerContenu(accueil, false)</code>	accueil	false	Masquer les cartes
<code>changerContenu(competences, false)</code>	competences	false	Masquer les cartes
<code>changerContenu(projets, false)</code>	projets	true	Afficher les cartes

Point fondamental

La propriété `projets.afficherProjets` ne devient jamais `false`. La fonction ne modifie aucune propriété des objets. C'est la référence reçue par le paramètre contenu qui change à chaque appel.

3.4 Ce que `const` garantit réellement

`const` empêche de réaffecter complètement la variable, mais ne transforme pas automatiquement l'objet en objet immuable. Le script actuel ne modifie pas ses propriétés.

```
accueil = {}; // interdit : réaffectation
accueil.titre = "Nouveau titre"; // techniquement possible
// Le projet ne fait aucune de ces deux opérations.
```

Autre subtilité

`sourcesImages` reçoit les trois chaînes au moment de la création du tableau. Si une propriété image était modifiée plus tard, le tableau ne se mettrait pas automatiquement à jour.

4. Fonctions, paramètres et arguments

```
function changerContenu(contenu, premierAffichage) {  
  // ...  
}
```

Une fonction est un bloc de code réutilisable. Un paramètre est un nom local écrit dans sa déclaration. Un argument est la valeur concrète envoyée lors d'un appel.

Appel	Valeur reçue par contenu	Valeur reçue par premierAffichage
<code>changerContenu(accueil, true)</code>	objet accueil	<code>true</code>
<code>changerContenu(accueil, false)</code>	objet accueil	<code>false</code>
<code>changerContenu(competences, false)</code>	objet competences	<code>false</code>
<code>changerContenu(projets, false)</code>	objet projets	<code>false</code>

4.1 Un paramètre peut représenter plusieurs objets

Pendant `changerContenu(projets, false)`, le paramètre `contenu` référence l'objet `projets`. Il n'est pas une copie modifiée et il n'est pas définitivement associé à `Projets`. Lors de l'appel suivant, un nouveau contexte local est créé avec une nouvelle valeur.

```
contenu.titre      // "Mes projets"  
contenu.image      // "images/liberté.jpg"  
contenu.afficherProjets // true  
premierAffichage  // false
```

4.2 Booléens et variable locale

Un booléen possède seulement deux valeurs : `true` ou `false`. `premierAffichage` est local à chaque appel. Sa valeur décide si la fonction doit remplir la page immédiatement ou lancer d'abord une transition de sortie.

Attention aux guillemets

`false` est un booléen. La valeur `"false"` est une chaîne non vide et elle est considérée comme vraie dans une condition. Ces deux valeurs ne sont donc pas interchangeables.

4.3 Cas limite révélateur

L'appel inhabituel `changerContenu(projets, true)` mettrait bien à jour le titre, le texte et l'image. Cependant, il n'ajouterait pas `visible` au bloc `Projets`, car cette instruction se trouve uniquement dans la branche `else`. Le code réserve donc en pratique la valeur `true` au chargement initial d'Accueil.

5. Explication complète des conditions

Règle fondamentale

Le code placé à l'intérieur d'un if ne s'exécute que si la condition évaluée vaut true.

```
if (premierAffichage) {  
  // Affichage immédiat  
} else {  
  // Changement animé  
}
```

Lors du chargement, `premierAffichage` vaut `true`: seule la première branche s'exécute. Lors d'un clic, il vaut `false`: la première branche est ignorée et `else` s'exécute.

5.1 Le point d'exclamation

```
!false // true  
!true  // false
```

Le point d'exclamation est l'opérateur de négation logique. Il produit la valeur inverse pour l'évaluation en cours. Il ne réécrit pas la valeur stockée dans l'objet.

```
if (!contenu.afficherProjets) {  
  blocsProjets.classList.add("cache-projets");  
}
```

Dans ce projet, puisque la propriété contient toujours un véritable booléen, cette condition équivaut à `contenu.afficherProjets === false`. La forme avec `!` est plus courte.

5.2 Démonstration avec les trois objets

Objet reçu	Valeur	Valeur inversée	Le bloc du if s'exécute ?
accueil	false	!false = true	Oui
competences	false	!false = true	Oui
projets	true	!true = false	Non

Accueil

```
contenu représente accueil  
accueil.afficherProjets vaut false  
!false vaut true  
la condition est vraie  
le code du if s'exécute
```

Compétences

```
contenu représente competences  
competences.afficherProjets vaut false  
!false vaut true  
la condition est vraie  
le code du if s'exécute
```

Projets

```
contenu représente projets  
projets.afficherProjets vaut true  
!true vaut false  
la condition est fausse  
le code du if est ignoré
```

5.3 Pourquoi deux if séparés ?

Le test négatif commence le masquage avant l'attente de 300 ms. Le test positif se trouve dans le deuxième callback et commence l'affichage après l'attente. Ils ne forment donc pas un simple `if...else` : ils interviennent à deux moments différents de l'animation.

6. Le DOM et les éléments HTML

Le DOM, pour Document Object Model, est la représentation en objets de la page HTML. Le navigateur transforme chaque balise en élément manipulable par JavaScript. L'objet global `document` représente le document courant.

```
const titre = document.getElementById("titreh2");
const paragraphe = document.getElementById("textep");
const image = document.getElementById("monImage");
const blocsProjets = document.getElementById("projetsLiens");
```

Constante JavaScript	Identifiant recherché	Élément HTML	Rôle
titre	titreh2	<h2 id="titreh2">	Titre dynamique
paragraphe	textep	<p id="textep">	Texte dynamique
image	monImage		Image dynamique
blocsProjets	projetsLiens	<div id="projetsLiens">	Cartes de projets

6.1 Ce que renvoie getElementById

`document.getElementById` est une méthode de l'objet `document`. Elle reçoit un identifiant HTML et renvoie la référence vers l'élément correspondant. La constante ne contient pas une copie du HTML : elle désigne l'élément vivant de la page.

Déclarer cette référence avec `const` interdit de faire pointer la variable vers un autre élément. Cela n'empêche pas de modifier le contenu, la source, le texte alternatif ou les classes de l'élément visé.

6.2 Les éléments sont initialement vides

```
<h2 id="titreh2"></h2>
<p id="textep"></p>
<img id="monImage" alt="">
```

Le HTML fournit les emplacements, mais pas leur contenu principal. C'est l'appel initial à `changerContenu(accueil, true)` qui les remplit. Cette organisation explique pourquoi une erreur JavaScript pourrait laisser une zone vide.

6.3 Le risque de null

Si l'identifiant demandé n'existe pas, `getElementById` renvoie `null`. Une instruction suivante essaierait alors d'utiliser une propriété sur une valeur qui ne représente aucun élément.

```
const titre = document.getElementById("mauvais-id");
titre.innerHTML = contenu.titre;
// TypeError : impossible de modifier innerHTML sur null
```

Diagnostic

Dans les outils de développement, lire le premier message rouge de la console, repérer le nom de la propriété et vérifier exactement la casse de l'identifiant dans `index.html`.

7. Différence entre fonction, méthode et constructeur

Une fonction est appelée directement par son nom. Une méthode est une fonction accessible à partir d'un objet avec un point. Un constructeur sert à créer un nouvel objet avec `new`.

Écriture	Catégorie	Objet concerné	Rôle
<code>prechargerImages()</code>	Fonction	Aucun objet à gauche	Lance le préchargement
<code>changerContenu(...)</code>	Fonction	Aucun objet à gauche	Orchestre l'interface
<code>mettreAJourContenu()</code>	Fonction interne	Aucun objet à gauche	Affecte les données
<code>sourcesImages.forEach(...)</code>	Méthode	Tableau	Parcourt les valeurs
<code>document.getElementById(...)</code>	Méthode	<code>document</code>	Recherche un élément
<code>element.classList.add(...)</code>	Méthode	<code>classList</code>	Ajoute une classe
<code>element.classList.remove(...)</code>	Méthode	<code>classList</code>	Retire une classe
<code>classList.toggle(...)</code>	Méthode	<code>classList</code>	Bascule une classe
<code>addEventListener(...)</code>	Méthode	Élément du DOM	Écoute un événement
<code>new Image()</code>	Constructeur	<code>Image</code>	Crée un objet image

7.1 Décomposer `element.classList.add`

```
element.classList.add("fade-in");
```

- `element` représente successivement le titre, le paragraphe puis l'image.
- `classList` est une propriété donnant accès à la liste de classes de cet élément.
- `add()` est une méthode de cette liste, dont le type est `DOMTokenList`.

7.2 Le cas de `setTimeout`

`setTimeout` est fourni par l'environnement du navigateur. On l'utilise ici comme une fonction globale, même s'il correspond aussi à une capacité de l'objet `window`. Il reçoit un callback et un délai minimal.

8. Fonction interne, portée et closure

La portée est la zone du code dans laquelle un nom est accessible. Les objets de contenu sont déclarés dans la portée globale du script. Les constantes créées dans `changerContenu` sont locales à chaque appel.

```
function changerContenu(contenu, premierAffichage) {  
  const titre = document.getElementById("titreh2");  
  const paragraphe = document.getElementById("textep");  
  const image = document.getElementById("monImage");  
  
  function mettreAJourContenu() {  
    titre.innerHTML = contenu.titre;  
    paragraphe.innerHTML = contenu.texte;  
    image.src = contenu.image;  
  }  
}
```

8.1 Pourquoi la fonction interne voit les variables

`mettreAJourContenu` est déclarée dans l'environnement de `changerContenu`. Elle conserve donc l'accès au paramètre `contenu` et aux constantes `titre`, `paragraphe` et `image`. Cette capacité est appelée fermeture lexicale, ou closure.

Image mentale

La fonction interne voyage avec un petit sac contenant les références de l'appel dans lequel elle a été créée. Même le callback exécuté plus tard peut encore retrouver ce sac.

8.2 Pourquoi ne pas passer quatre arguments ?

La fonction interne n'a besoin d'exister que pour organiser une opération locale. Lui transmettre à chaque appel les quatre valeurs déjà disponibles rendrait le code plus lourd sans apporter d'information. Limiter sa portée empêche aussi de l'appeler depuis un endroit où son contexte n'existerait pas.

8.3 Une closure par appel

Chaque clic crée un nouvel appel, donc de nouvelles constantes locales et une nouvelle fonction `mettreAJourContenu`. Si plusieurs clics créent plusieurs temporisations, chaque callback conserve les données de son propre appel. Cette propriété explique à la fois le bon fonctionnement des délais et le risque de concurrence lors de clics très rapides.

9. Mise à jour du contenu

```
titre.innerHTML = contenu.titre;  
paragraphe.innerHTML = contenu.texte;  
image.src = contenu.image;  
image.alt = `Illustration de ${contenu.titre}`;
```

Le signe `=` réalise une affectation : la valeur située à droite est calculée, puis placée dans la propriété située à gauche. Ces quatre lignes forment le point précis où les données JavaScript deviennent visibles dans la page.

9.1 innerHTML et contenu.texte

- `titre.innerHTML` représente le contenu HTML interne du titre déjà présent dans la page.
- `contenu.titre` est la chaîne stockée dans l'objet reçu.
- `paragraphe.innerHTML` reçoit le texte long et interprète ses balises `<br>`.
- `textContent` afficherait les caractères des balises comme du texte au lieu de créer ces retours à la ligne.

Pourquoi innerHTML est acceptable ici

Les chaînes viennent directement du fichier `script.js` et sont contrôlées par le développeur. Si elles venaient d'un utilisateur ou d'une API non fiable, `innerHTML` pourrait permettre une injection et il faudrait privilégier `textContent` ou assainir les données.

9.2 src et alt

`image.src` indique au navigateur quelle ressource doit être affichée dans l'élément `img`. `image.alt` fournit une alternative textuelle utile aux lecteurs d'écran et lorsque l'image ne peut pas être chargée.

Les backticks créent un template literal. L'expression placée entre `${` et `}` est évaluée et insérée dans la chaîne.

Objet	Valeur finale de image.alt
<code>accueil</code>	Illustration de Bienvenue sur mon portfolio
<code>competences</code>	Illustration de Mes compétences
<code>projets</code>	Illustration de Mes projets

9.3 Flux complet des données

```
Clic sur un lien  
  |  
  v  
Objet envoyé en argument  
  |  
  v  
Paramètre contenu  
  |  
  v  
mettreAJourContenu()  
  |  
  +--> innerHTML du titre  
  +--> innerHTML du paragraphe  
  +--> src et alt de l'image  
  |  
  v  
Nouvel affichage dans le navigateur
```


10. Tableaux, forEach et callbacks

```
const elementsAnimes = [titre, paragraphe, image];
```

Un tableau est une collection ordonnée. Les crochets contiennent ici trois références vers des éléments du DOM. L'ordre est le titre, le paragraphe, puis l'image. Une constante peut parfaitement contenir un tableau : elle interdit de réaffecter la variable, mais le tableau reste un objet.

Indice	Valeur	Élément HTML
0	titre	<h2 id="titreh2">
1	paragraphe	<p id="textep">
2	image	

10.1 Déroulement de forEach

```
elementsAnimes.forEach((element) => {  
  element.classList.add("fade-in");  
});
```

forEach est une méthode du tableau. Elle appelle le callback une fois pour chaque valeur, dans l'ordre. Un passage s'appelle une itération. Le paramètre local **element** reçoit successivement le titre, le paragraphe et l'image.

Itération	Valeur de element	Action
1	titre	Ajoute fade-in au h2
2	paragraphe	Ajoute fade-in au p
3	image	Ajoute fade-in au img

10.2 Fonction fléchée et callback

La syntaxe **(element) => { ... }** déclare une fonction fléchée. Elle devient un callback parce qu'elle est transmise à **forEach**, qui décide quand l'appeler. Ici, **forEach** est synchrone : les trois appels ont lieu immédiatement, l'un après l'autre. Les trois animations commencent néanmoins dans la même phase de rendu du navigateur.

Même syntaxe, moments différents

Un callback de **forEach** est appelé pendant la boucle. Un callback de **setTimeout** est appelé plus tard. Un callback d'événement attend un clic. La flèche seule ne détermine pas le moment d'exécution.

10.3 Pourquoi éviter la répétition

```
titre.classList.add("fade-in");  
paragraphe.classList.add("fade-in");  
image.classList.add("fade-in");
```

Cette version fonctionnerait. Le tableau devient plus pratique lorsque la même opération doit être appliquée plusieurs fois. Si un quatrième élément devait un jour partager l'animation, il suffirait de l'ajouter au tableau au lieu de dupliquer chaque bloc.

10.4 Le tableau des sources

```
const sourcesImages = [  
  accueil.image,  
  competences.image,  
  projets.image  
];
```

Au moment de sa création, le tableau contient trois chaînes : `images/villedenuit1.jpg`, `images/competences-v2.png` et `images/liberté.jpg`. Ces valeurs primitives sont copiées dans le tableau à cet instant.

11. Classes CSS et classList

L'attribut HTML `class` contient des noms utilisés par le CSS. JavaScript peut modifier cette liste avec la propriété `classList`. Il ne calcule pas lui-même chaque valeur d'opacité : il change l'état, puis le moteur CSS applique la règle correspondante.

```
element.classList.add("fade-in");
element.classList.remove("fade-in");
element.classList.add("fade-out");
element.classList.remove("fade-out");
```

Méthode	Effet
<code>add("nom")</code>	Ajoute la classe si elle n'existe pas déjà
<code>remove("nom")</code>	Retire la classe si elle existe
<code>toggle("nom")</code>	Ajoute une classe absente ou retire une classe présente

11.1 Les fondus du contenu principal

```
@keyframes fadeOut {
  from { opacity: 1; }
  to   { opacity: 0; }
}

.fade-out { animation: fadeOut 0.3s forwards; }
.fade-in  { animation: fadeIn 2s forwards; }
```

Le fondu sortant du titre, du texte et de l'image dure 0,3 seconde. Leur fondu entrant dure réellement 2 secondes. Le mot `forwards` conserve visuellement la dernière valeur d'opacité après la fin de l'animation tant que la classe reste présente.

11.2 Le bloc Projets

```
.projets-liens {
  display: none;
}

.projets-liens.visible {
  display: flex;
  animation: fadeIn 0.3s forwards;
}

.projets-liens.cache-projets {
  animation: fadeOut 0.3s forwards;
}
```

Par défaut, les cartes ne participent pas à la mise en page. La classe `visible` les affiche en flexbox. Pour les masquer avec un fondu, le script ajoute d'abord `cache-projets` tout en conservant temporairement `visible`.

Pourquoi les deux classes coexistent

`visible` maintient `display: flex`. `cache-projets` applique `fadeOut`. Les deux sélecteurs ont la même spécificité, mais `cache-projets` est déclaré après `visible` et son animation gagne. Après 300 ms, retirer `visible` rend de nouveau `display: none`.

11.3 Pourquoi retirer visible trop tôt casse l'effet

```
blocsProjets.classList.add("cache-projets");  
blocsProjets.classList.remove("visible"); // trop tôt
```

Cette version rétablit immédiatement `display: none`. Or la propriété `display` ne se fonde pas progressivement : le bloc disparaît de la mise en page et l'animation d'opacité ne peut plus être observée.

12. Explication approfondie de setTimeout

```
setTimeout(() => {  
  // Code exécuté plus tard  
}, 300);
```

`setTimeout` demande au navigateur de rendre un callback éligible après un délai minimal. Le nombre 300 représente 300 millisecondes, soit 0,3 seconde, car 1000 millisecondes forment une seconde.

Pas une horloge exacte

Le callback n'est jamais garanti exactement à 300 ms. Il peut s'exécuter plus tard si JavaScript est occupé. La formulation exacte est : au plus tôt après le délai indiqué.

12.1 Pourquoi le programme n'est pas bloqué

Le navigateur enregistre le callback et laisse la fonction courante continuer. Pendant l'attente, le navigateur peut calculer les styles, peindre les images intermédiaires de l'animation, répondre à d'autres événements et poursuivre les chargements réseau. Quand le délai est écoulé et que la pile JavaScript est libre, le callback rejoint le travail à exécuter.

```
Appel de setTimeout  
  |  
  +--> enregistrement du callback et du délai  
  |  
  v  
la fonction courante continue et se termine  
  |  
  v  
après au moins 300 ms, callback prêt  
  |  
  v  
exécution dès que JavaScript est disponible
```

12.2 Premier délai : masquer le bloc Projets

```
if (!contenu.afficherProjets) {  
  blocsProjets.classList.add("cache-projets");  
  
  setTimeout(() => {  
    blocsProjets.classList.remove("visible");  
    blocsProjets.classList.remove("cache-projets");  
  }, 300);  
}
```

- 1 À 0 ms, cache-projets est ajoutée. Si le bloc venait de Projets, visible reste présente et maintient display: flex.
- 2 Entre 0 et environ 300 ms, l'opacité du bloc diminue grâce à fadeOut.
- 3 Après au moins 300 ms, visible est retirée, ce qui rétablit display: none.
- 4 cache-projets est ensuite retirée afin de nettoyer l'état pour un futur affichage.

12.3 Deuxième délai : remplacer le contenu

```
setTimeout(() => {
  mettreAJourContenu();

  if (contenu.afficherProjets) {
    blocsProjets.classList.add("visible");
  }

  elementsAnimes.forEach((element) => {
    element.classList.remove("fade-out");
    element.classList.add("fade-in");
  });
}, 300);
```

1	À 0 ms, l'ancien titre, l'ancien texte et l'ancienne image reçoivent fade-out.
2	Pendant environ 300 ms, l'ancien contenu disparaît progressivement.
3	Après au moins 300 ms, les quatre propriétés du DOM sont remplacées.
4	Si le nouvel objet demande les projets, visible est ajoutée à ce même moment.
5	fade-out est retirée et fade-in est ajoutée aux trois éléments principaux.

L'ajout de **visible** dans ce callback permet au bloc Projets et au nouveau contenu de commencer leur entrée ensemble. Ils ne finissent toutefois pas ensemble : 0,3 seconde pour les cartes et 2 secondes pour le titre, le texte et l'image.

12.4 Troisième délai : fermer la navigation

```
navigationMobile.addEventListener("click", () => {
  setTimeout(() => {
    navigationMobile.classList.remove("ouvert");
  }, 300);
});
```

Ce délai est bien présent dans le code réel. En revanche, aucune animation CSS de fermeture du menu ne correspond à ces 300 ms. Sur un écran responsive, le menu reste donc affiché pendant ce délai, puis passe directement de **display: block** à **display: none**.

13. Préchargement des images

```
function prechargerImages() {  
  sourcesImages.forEach((sourceImage) => {  
    const imagePrechargee = new Image();  
    imagePrechargee.src = sourceImage;  
  });  
}
```

`new Image()` crée un objet image en mémoire. Aucun appel comme `appendChild` ne l'insère dans le document : cette image n'est donc pas une nouvelle balise visible. L'affectation à `src` suffit à demander la ressource au navigateur.

13.1 Les trois itérations

- | | |
|---|---|
| 1 | <code>sourceImage</code> reçoit <code>images/villedenuit1.jpg</code> , puis un premier objet <code>Image</code> est créé. |
| 2 | <code>sourceImage</code> reçoit <code>images/competences-v2.png</code> , puis un deuxième objet est créé. |
| 3 | <code>sourceImage</code> reçoit <code>images/liberté.jpg</code> , puis un troisième objet est créé. |

La constante `imagePrechargee` est recrée à chaque itération dans une portée locale. Elle n'a pas besoin d'être utilisée ailleurs : son rôle est simplement de recevoir une source afin de lancer le travail du navigateur.

13.2 Cache, réseau et limite du code

Le navigateur peut télécharger l'image et la conserver dans son cache. Quand l'image visible reçoit ensuite le même chemin, il peut réutiliser une requête déjà en cours ou la ressource en cache, ce qui réduit le clignotement.

Nuance indispensable

La fonction ne contient ni événement `load`, ni promesse, ni `await`. Elle lance les trois demandes puis se termine. L'appel suivant n'attend donc pas que tous les fichiers soient entièrement téléchargés ou décodés.

```
prechargerImages()  
  |  
  +--> lance requête Accueil  
  +--> lance requête Compétences  
  +--> lance requête Projets  
  |  
  v  
retour de la fonction sans attente  
  |  
  v  
changerContenu(accueil, true)
```

14. Le premier affichage

```
prechargerImages();  
changerContenu(accueil, true);
```

L'appel de préchargement est placé en premier afin de démarrer les requêtes le plus tôt possible. L'appel suivant remplit immédiatement les éléments vides du HTML avec les données d'Accueil.

- | | |
|---|--|
| 1 | La fonction recherche le titre, le paragraphe, l'image et le bloc Projets. |
| 2 | <code>premierAffichage</code> vaut <code>true</code> , donc la première branche du <code>if</code> s'exécute. |
| 3 | <code>mettreAJourContenu()</code> affecte immédiatement le titre, le texte, <code>src</code> et <code>alt</code> . |
| 4 | La boucle ajoute fade-in au titre, au paragraphe et à l'image. |
| 5 | Le CSS les fait passer d'une opacité de 0 à 1 pendant 2 secondes. |
| 6 | Aucun <code>setTimeout</code> de <code>changerContenu</code> n'est créé et aucune classe du bloc Projets n'est modifiée. |

14.1 Pourquoi aucun fade-out ?

Au chargement, le titre et le paragraphe sont vides et l'image n'a pas encore de source. Il n'existe donc aucun ancien contenu utile à faire disparaître. Attendre 300 ms créerait seulement un retard sans transition visible.

Différence essentielle

`true` signifie ici initialisation directe. `false` signifie changement demandé après le premier affichage. Il ne s'agit pas de dire si Accueil est vrai ou faux.

15. Gestion du menu burger

```
const boutonBurger = document.getElementById("menuBurger");
const navigationMobile = document.getElementById("menuNavigation");
```

Le bouton HTML possède l'identifiant `menuBurger`. La variable `navigationMobile` désigne le seul élément `nav` de la page. Il ne s'agit pas d'une seconde navigation : le CSS transforme cette même navigation selon la largeur.

15.1 Ouvrir et fermer avec toggle

```
boutonBurger.addEventListener("click", () => {
  navigationMobile.classList.toggle("ouvert");
});
```

Un événement est un signal produit par le navigateur. Ici, `click` signifie que le bouton a été activé. `addEventListener` enregistre un callback qui reste en attente. Au premier clic, `toggle` ajoute `ouvert`; au clic suivant, il la retire.

Largeur	Burger	Navigation sans ouvert	Avec nav.ouvert
Plus de 1100 px	Masqué	Navigation normale visible	Pas d'effet responsive utile
769 à 1100 px	Visible	<code>display: none</code>	<code>display: block</code>
768 px ou moins	Visible	<code>display: none</code>	<code>display: block</code>

15.2 Fermer après un clic dans la navigation

```
navigationMobile.addEventListener("click", () => {
  setTimeout(() => {
    navigationMobile.classList.remove("ouvert");
  }, 300);
});
```

L'écouteur est placé sur le `nav`, pas directement sur chaque lien. Un clic sur un lien commence sur l'élément `a` puis remonte dans ses parents jusqu'au `nav`. Cette remontée s'appelle la propagation de l'événement.

```
<a> -> <li> -> <ul> -> <nav> -> parents
```

Le `return false` de l'attribut `onclick` annule le comportement de `href="#"`, qui pourrait modifier le fragment ou remonter la page. Il n'arrête pas la propagation : l'écouteur du `nav` reçoit donc toujours le clic.

Comportement exact

Tout clic reçu par le `nav` ou l'un de ses descendants programme la fermeture, pas uniquement un clic sur un véritable lien. Sur ordinateur, retirer ouvert n'a pas d'effet visuel.

15.3 Chronologie du menu

Ouverture

1	L'utilisateur clique sur le bouton burger.
2	Le callback de l'écouteur s'exécute.
3	toggle ajoute ouvert si la classe est absente.
4	Dans les media queries responsive, nav.ouvert applique display: block.
5	Le menu apparaît immédiatement, sans animation CSS dédiée.

Fermeture après sélection

1	Le lien appelle changerContenu avec l'objet choisi et false.
2	Le clic remonte ensuite jusqu'à la navigation.
3	L'écouteur du nav programme un retrait après 300 ms minimum.
4	Le menu reste affiché pendant ce délai.
5	ouvert est retirée et le menu redevient display: none immédiatement.

15.4 Accessibilité actuelle

Le bouton possède déjà un `aria-label` et `aria-controls="menuNavigation"`. En revanche, le script ne met pas à jour `aria-expanded`. Un lecteur d'écran ne reçoit donc pas explicitement l'état ouvert ou fermé. Cette amélioration sera présentée plus loin, sans être appliquée au projet.

16. Chronologies exactes des interactions

Dans les chronologies suivantes, 300 ms signifie au plus tôt après 300 ms. Les durées CSS sont nominales et peuvent sembler légèrement différentes selon la charge du navigateur.

16.1 Clic sur Projets

1	Le lien appelle <code>changerContenu(projets, false)</code> .
2	<code>contenu.referenceProjets</code> et <code>contenu.afficherProjets</code> vaut <code>true</code> .
3	<code>fade-in</code> est retirée, puis <code>fade-out</code> est ajoutée au titre, au paragraphe et à l'image.
4	Leur <code>fade-out</code> de 0,3 seconde commence.
5	La condition <code>!contenu.afficherProjets</code> vaut <code>false</code> : aucun timer de masquage des cartes n'est créé.
6	Après au moins 300 ms, le titre, le texte, <code>src</code> et <code>alt</code> sont remplacés.
7	<code>visible</code> est ajoutée au bloc Projets.
8	<code>fade-out</code> est retirée et <code>fade-in</code> est ajoutée aux trois éléments principaux.
9	Les cartes entrent pendant 0,3 seconde et le contenu principal pendant 2 secondes.

```
0 à environ 300 ms : ancien contenu en fade-out
à partir de 300 ms : nouvelles données insérées
300 à 600 ms      : bloc Projets en fade-in
300 à 2300 ms     : titre, texte et image en fade-in
```

16.2 Clic sur Accueil

1	Le lien appelle <code>changerContenu(accueil, false)</code> .
2	<code>contenu.afficherProjets</code> vaut <code>false</code> et sa négation vaut <code>true</code> .
3	Le contenu principal commence son <code>fade-out</code> .
4	<code>cache-projets</code> est ajoutée, même si le bloc est déjà caché.
5	Un premier timer programme le retrait de <code>visible</code> et <code>cache-projets</code> .
6	Un second timer programme la mise à jour du contenu.
7	Après au moins 300 ms, les cartes sont nettoyées et le contenu Accueil est affecté.
8	Le nouveau titre, le nouveau texte et la nouvelle image entrent pendant 2 secondes.

16.3 Clic sur Compétences

La chronologie est identique à celle d'Accueil car `competences.afficherProjets` vaut `false`. Seules les données affectées après le délai changent : titre Mes compétences, texte correspondant, image `images/competences-v2.png` et alternative Illustration de Mes compétences.

16.4 Quitter Projets

1	État initial : le bloc possède projets-liens et visible, donc display: flex.
2	Un clic sur Accueil ou Compétences ajoute cache-projets sans retirer visible.
3	visible maintient le bloc dans la mise en page pendant que cache-projets lance fadeOut.
4	Le bloc et les trois éléments principaux disparaissent simultanément pendant 0,3 seconde.
5	Après au moins 300 ms, visible est retirée : le bloc retrouve display: none.
6	cache-projets est retirée, le nouveau contenu est affecté, puis son fade-in de 2 secondes commence.

16.5 Clics rapides et callbacks concurrents

Chaque clic crée de nouvelles closures et de nouveaux timers. Aucun identifiant de timer n'est mémorisé et aucun `clearTimeout` n'annule le travail précédent.

```
t = 0 ms      clic sur Accueil
t = 100 ms    clic sur Projets
t >= 300 ms  ancien callback affiche provisoirement Accueil
t >= 400 ms  callback plus récent affiche Projets
```

- Un contenu intermédiaire peut apparaître brièvement.
- Une ancienne temporisation peut ajouter ou retirer une classe au mauvais moment.
- Ajouter une classe déjà présente ne redémarre pas forcément son animation.
- Un ancien timer du menu peut refermer un menu que l'utilisateur vient de rouvrir.

Limite, pas faute de syntaxe

Le code est adapté à une navigation normale. La gestion des clics très rapides est une amélioration possible, pas une fonctionnalité déjà présente.

17. Tableau récapitulatif des notions

Ce tableau relie chaque définition à une écriture réellement présente dans le projet. Les catégories fonction, méthode, propriété et constructeur sont volontairement distinguées.

Notion	Exemple du projet	Explication
Objet	<code>accueil</code>	Ensemble de propriétés regroupant les données d'une section
Objet littéral	<code>const accueil = { ... }</code>	Objet écrit directement avec des accolades
Propriété	<code>titre</code>	Information rattachée à un objet
Valeur	<code>"Mes projets"</code>	Contenu affecté à une propriété
Constante	<code>const projets</code>	Liaison qui ne peut pas être réaffectée
Chaîne	<code>"Mes compétences"</code>	Valeur textuelle
Template literal	<code>`...`</code>	Chaîne avec backticks, multiligne et interpolable
Interpolation	<code>\${contenu.titre}</code>	Insertion d'une expression dans une chaîne
Booléen	<code>true / false</code>	Valeur logique utilisée pour prendre une décision
Tableau	<code>elementsAnimes</code>	Collection ordonnée de plusieurs valeurs
Fonction	<code>changerContenu</code>	Bloc de code réutilisable appelé directement
Fonction interne	<code>mettreAJourContenu</code>	Fonction limitée à la portée de <code>changerContenu</code>
Méthode	<code>classList.add</code>	Fonction rattachée à un objet
Constructeur	<code>Image</code>	Fonction utilisée avec <code>new</code> pour créer un objet
Paramètre	<code>contenu</code>	Nom local écrit dans la déclaration d'une fonction
Argument	<code>projets</code>	Valeur concrète envoyée lors d'un appel
Variable locale	<code>titre</code>	Nom accessible dans l'appel courant de <code>changerContenu</code>
Portée	<code>{ ... }</code>	Zone dans laquelle un nom est accessible
Closure	<code>mettreAJourContenu</code>	Fonction conservant l'accès à son environnement lexical
Condition	<code>if (premierAffichage)</code>	Choix d'une branche selon une valeur
Négation	<code>!contenu.afficherProjets</code>	Inversion temporaire d'une valeur logique
Affectation	<code>image.src = contenu.image</code>	Placement d'une valeur dans une propriété
DOM	<code>document</code>	Représentation JavaScript de la page
Identifiant	<code>titreh2</code>	Nom HTML unique utilisé pour retrouver un élément
Élément DOM	<code>titre</code>	Objet représentant une balise réelle de la page
Propriété DOM	<code>innerHTML</code>	Donnée lisible ou modifiable sur un élément
classList	<code>element.classList</code>	Liste manipulable des classes d'un élément
Itération	Un passage de <code>forEach</code>	Traitement d'une valeur du tableau
Fonction fléchée	<code>(element) => { ... }</code>	Syntaxe courte utilisée pour les callbacks
Callback	Fonction de <code>setTimeout</code>	Fonction transmise pour être exécutée par un autre mécanisme
Temporisation	<code>setTimeout(..., 300)</code>	Programmation d'un callback après un délai minimal

Notion	Exemple du projet	Explication
Événement	<code>"click"</code>	Signal produit par le navigateur
Écouteur	<code>addEventListener</code>	Enregistrement d'une réaction à un événement
Propagation	Clic du lien vers le nav	Remontée de l'événement dans les parents du DOM
Cache	<code>new Image()</code>	Réutilisation possible d'une ressource déjà demandée
Valeur null	Résultat possible de <code>getElementById</code>	Absence d'élément correspondant

18. Lecture complète de script.js, bloc par bloc

Cette lecture reprend le fichier dans son ordre réel. Les textes très longs des objets sont abrégés dans les extraits, mais leur rôle et leurs balises sont expliqués intégralement. Les commentaires commençant par `//` sont destinés au lecteur et ne sont pas exécutés.

18.1 Déclaration des trois objets

```
const accueil = { titre: "Bienvenue sur mon portfolio", ... };  
const competences = { titre: "Mes compétences", ... };  
const projets = { titre: "Mes projets", ... };
```

- Chaque `const` crée une liaison globale dans ce script classique.
- Chaque expression entre accolades construit immédiatement un objet.
- Les textes entre backticks peuvent occuper plusieurs lignes et conserver les balises `br`.
- Les chemins d'images sont relatifs à la page `index.html`.
- Les booléens sont lus plus tard par `changerContenu` ; aucune classe n'est encore modifiée.

Moment et résultat

Ce bloc s'exécute dès le début de `script.js`. Il crée les données en mémoire, sans effet visuel immédiat. Il n'accède pas encore au DOM ni au CSS.

18.2 Création de `sourcesImages`

```
const sourcesImages = [ accueil.image, competences.image, projets.image];
```

Les trois objets existent déjà, donc leurs propriétés `image` peuvent être lues. Le tableau reçoit trois chaînes dans l'ordre. Il sera accessible à `prechargerImages` depuis la portée extérieure. Aucun téléchargement ne commence à cette ligne : seul le tableau est créé.

18.3 Déclaration et appel du préchargement

```
function prechargerImages() {  
  sourcesImages.forEach((sourceImage) => {  
    const imagePrechargee = new Image();  
    imagePrechargee.src = sourceImage;  
  });  
}
```

Ligne	Explication précise
<code>function prechargerImages()</code>	Déclare une fonction globale sans l'exécuter
<code>sourcesImages.forEach</code>	Parcourt immédiatement les trois chaînes quand la fonction sera appelée
<code>(sourceImage) =></code>	Déclare le callback et son paramètre local
<code>new Image()</code>	Construit un nouvel objet image non inséré dans le HTML
<code>imagePrechargee.src = sourceImage</code>	Affecte le chemin et peut déclencher la requête

Accès et effet

La fonction accède au tableau global `sourcesImages`. Le callback accède à `sourceImage` et à sa propre constante `imagePrechargee`. L'effet est réseau/cache, pas une nouvelle image visible.

18.4 Entrée de changerContenu et sélection du DOM

```
function changerContenu(contenu, premierAffichage) {  
  const titre = document.getElementById("titreh2");  
  const paragraphe = document.getElementById("textep");  
  const image = document.getElementById("monImage");  
  const blocsProjets = document.getElementById("projetsLiens");  
  const elementsAnimes = [titre, paragraphe, image];
```

Ce bloc est exécuté à chaque appel, y compris au chargement initial. Les quatre recherches sont donc refaites après chaque clic. Les constantes sont locales à l'appel courant. `elementsAnimes` regroupe seulement les trois éléments qui partagent les classes fade-in et fade-out.

Liens HTML

Les identifiants doivent correspondre exactement à `titreh2`, `textep`, `monImage` et `projetsLiens` dans `index.html`. Si l'un manque, le tableau peut contenir `null` et `classList` provoquera une erreur.

18.5 Fonction interne de mise à jour

```
function mettreAJourContenu() {  
  titre.innerHTML = contenu.titre;  
  paragraphe.innerHTML = contenu.texte;  
  image.src = contenu.image;  
  image.alt = `Illustration de ${contenu.titre}`;  
}
```

Affectation	Résultat visible ou accessible
<code>titre.innerHTML</code>	Remplace le texte du h2
<code>paragraphe.innerHTML</code>	Remplace le paragraphe et interprète les br
<code>image.src</code>	Change la ressource affichée
<code>image.alt</code>	Actualise l'alternative textuelle

Accès

Cette fonction interne accède par closure à `contenu`, `titre`, `paragraphe` et `image`. Elle ne connaît pas directement `premierAffichage` et n'a pas besoin de `blocsProjets`.

18.6 Branche du premier affichage

```
if (premierAffichage) {  
  mettreAJourContenu();  
  elementsAnimes.forEach((element) => {  
    element.classList.add("fade-in");  
  });  
}
```

Ce code ne s'exécute que pour l'appel initial avec `true`. La mise à jour précède l'ajout des classes : le navigateur dispose donc déjà des nouvelles données lorsque le fade-in commence. Le CSS applique une durée de 2 secondes. Aucun bloc Projets n'est affiché dans cette branche.

18.7 Début de la branche animée

```
else {
  elementsAnimes.forEach((element) => {
    element.classList.remove("fade-in");
    element.classList.add("fade-out");
  });
}
```

Cette branche s'exécute lors de chaque clic de navigation. Retirer l'ancienne classe avant d'ajouter la nouvelle évite de conserver deux animations concurrentes sur les mêmes éléments. Le navigateur associe immédiatement les éléments à la règle CSS `.fade-out`.

18.8 Masquage conditionnel des projets

```
if (!contenu.afficherProjets) {
  blocsProjets.classList.add("cache-projets");
  setTimeout(() => {
    blocsProjets.classList.remove("visible");
    blocsProjets.classList.remove("cache-projets");
  }, 300);
}
```

Le test porte sur le nouvel objet demandé. Pour Accueil et Compétences, le bloc doit être absent après la transition. Le callback conserve l'accès à `blocsProjets` grâce à la closure. Son résultat visible dépend des règles `.visible` et `.cache-projets`.

18.9 Remplacement différé et entrée

```
setTimeout(() => {
  mettreAJourContenu();
  if (contenu.afficherProjets) {
    blocsProjets.classList.add("visible");
  }
  elementsAnimes.forEach((element) => {
    element.classList.remove("fade-out");
    element.classList.add("fade-in");
  });
}, 300);
```

Le contenu n'est remplacé qu'après la durée nominale de sortie. Le test positif affiche les cartes uniquement pour Projets. Les trois éléments sont ensuite replacés dans l'état d'entrée. Visuellement, les nouvelles données ne sont donc pas visibles pendant le fade-out de l'ancien contenu.

18.10 Initialisation et menu

```
prechargerImages();
changerContenu(accueil, true);

const boutonBurger = document.getElementById("menuBurger");
const navigationMobile = document.getElementById("menuNavigation");

boutonBurger.addEventListener("click", () => {
  navigationMobile.classList.toggle("ouvert");
});
```

Les deux appels s'exécutent dès le chargement. Les deux dernières constantes sont ensuite créées dans la portée globale du script, puis le callback du bouton est enregistré sans être exécuté. Le CSS ne rend le bouton visible que sur tablette et mobile.

```
navigationMobile.addEventListener("click", () => {  
  setTimeout(() => {  
    navigationMobile.classList.remove("ouvert");  
  }, 300);  
});
```

Ce dernier bloc termine l'exécution initiale en enregistrant un écouteur. Le callback extérieur attend un clic reçu par la navigation ; le callback intérieur attend ensuite 300 ms minimum.

19. Schémas de fonctionnement

Ces schémas condensent les mécanismes étudiés. Ils peuvent être utilisés comme fiches de révision.

19.1 Chargement initial

```
Chargement de la page
  |
  v
Création de accueil, competences et projets
  |
  v
Création de sourcesImages
  |
  v
Déclaration des fonctions
  |
  v
prechargerImages()
  |
  v
changerContenu(accueil, true)
  |
  v
Mise à jour immédiate + fade-in
```

19.2 Mise à jour du contenu

```
contenu.titre -----> titre.innerHTML
contenu.texte -----> paragraphe.innerHTML
contenu.image -----> image.src
contenu.titre -----> image.alt interpolé
```

19.3 Clic sur Projets

```
Clic Projets
  |
  v
changerContenu(projets, false)
  |
  v
fade-out du contenu actuel
  |
  v
attente minimale de 300 ms
  |
  v
nouvelles données + visible
  |
  v
fade-in des cartes et du contenu
```

19.4 Clic sur Accueil après Projets

```
Clic Accueil
  |
  v
!accueil.afficherProjets = true
  |
  v
ajout de cache-projets sans retirer visible
  |
  v
fade-out pendant environ 300 ms
  |
  v
retrait de visible et cache-projets
  |
  v
données Accueil + fade-in
```

19.5 Clic sur Compétences

```
Clic Compétences
  |
  v
changerContenu(competences, false)
  |
  v
masquage éventuel des projets + fade-out
  |
  v
attente minimale de 300 ms
  |
  v
données Compétences + fade-in de 2 s
```

19.6 Préchargement et menu

Préchargement

```
sourcesImages
  |
  v
forEach : une valeur à la fois
  |
  v
new Image() + affectation de src
  |
  v
requête lancée, sans attente de load
```

Ouverture du menu

```
Clic burger -> toggle ouvert -> nav.ouvert -> display: block
```

Fermeture du menu

```
Clic dans nav -> propagation -> setTimeout 300 ms
  -> remove ouvert -> display: none
```

20. Subtilités, erreurs possibles et diagnostic

Chaque cas ci-dessous distingue l'erreur, son symptôme, la manière de la diagnostiquer et une correction possible. Les exemples ne sont pas appliqués au projet.

20.1 Mauvais identifiant DOM

Exemple	<code>document.getElementById("titreH2")</code> au lieu de <code>"titreh2"</code>
Symptôme	<code>getElementById</code> renvoie null, puis <code>innerHTML</code> ou <code>classList</code> déclenche une <code>TypeError</code> .
Diagnostic	Afficher la variable dans la console et comparer exactement l'identifiant dans <code>index.html</code> .
Correction	Faire correspondre la casse et l'orthographe des deux fichiers.

20.2 Script exécuté trop tôt

Exemple	Déplacer <code>script.js</code> dans le <code>head</code> sans <code>defer</code> ni attente
Symptôme	Plusieurs sélections renvoient null au chargement.
Diagnostic	Vérifier l'ordre du document et la ligne indiquée par la console.
Correction	Conserver le script après le HTML, utiliser <code>defer</code> ou attendre <code>DOMContentLoaded</code> .

20.3 Chaîne à la place d'un booléen

Exemple	<code>changerContenu(accueil, "false")</code>
Symptôme	La branche du premier affichage s'exécute car une chaîne non vide est vraie en contexte conditionnel.
Diagnostic	Inspecter <code>typeof premierAffichage</code> et afficher sa valeur.
Correction	Utiliser <code>false</code> sans guillemets.

20.4 Point d'exclamation oublié ou mal compris

Exemple	<code>if (contenu.afficherProjets)</code> autour du bloc de masquage
Symptôme	Les cartes seraient masquées lorsque Projets est demandé, donc au moment opposé.
Diagnostic	Écrire les trois valeurs dans une table de vérité.
Correction	Conserver la négation ou comparer explicitement à <code>false</code> .

20.5 Retrait de visible trop tôt

Exemple	Ajouter <code>cache-projets</code> puis retirer visible sans délai
Symptôme	Le bloc disparaît instantanément et son fade-out n'est pas observable.
Diagnostic	Inspecter les styles calculés : <code>display</code> redevient <code>none</code> immédiatement.
Correction	Retirer visible seulement après la durée de sortie.

20.6 Durées JavaScript et CSS désynchronisées

Exemple	Passer fade-out à 0,6 s en CSS en gardant <code>setTimeout</code> à 300
Symptôme	Le contenu est remplacé au milieu de la disparition.
Diagnostic	Comparer la durée de <code>.fade-out</code> et le délai du timer.
Correction	Maintenir les valeurs cohérentes ou écouter <code>animationend</code> .

20.7 Mauvais nom de classe

Exemple	<code>classList.add("fadein")</code> au lieu de <code>"fade-in"</code>
Symptôme	Aucune règle CSS ne correspond et l'animation ne démarre pas.
Diagnostic	Observer l'attribut <code>class</code> et rechercher le sélecteur dans <code>style.css</code> .
Correction	Employer exactement le même nom dans JavaScript et CSS.

20.8 Ancienne classe non retirée

Exemple	Ajouter <code>fade-in</code> en laissant <code>fade-out</code>
Symptôme	Deux règles d'animation peuvent se concurrencer ; l'ordre CSS décide de celle qui gagne.
Diagnostic	Observer <code>classList</code> pendant la transition.
Correction	Retirer explicitement l'ancien état avant d'ajouter le nouveau.

20.9 Fonction déclarée dans une mauvaise portée

Exemple	Essayer d'appeler <code>mettreAJourContenu</code> depuis la portée globale
Symptôme	<code>ReferenceError</code> , car la fonction interne n'existe que dans <code>changerContenu</code> .
Diagnostic	Vérifier les accolades et l'endroit de la déclaration.
Correction	Conserver l'appel dans sa portée ou transmettre explicitement les données si le besoin change.

20.10 Chemin d'image incorrect

Exemple	<code>image: "Images/Liberte.jpg"</code>
Symptôme	L'image est absente et la requête renvoie souvent 404.
Diagnostic	Ouvrir l'onglet Réseau et comparer chemin, extension, accents et casse.
Correction	Utiliser le chemin réel <code>images/liberté.jpg</code> .

20.11 Fichier absent du dépôt

Exemple	Le chemin est correct localement mais l'image n'a pas été ajoutée à Git
Symptôme	Le site fonctionne sur la machine puis échoue après déploiement.
Diagnostic	Vérifier les fichiers suivis et la réponse réseau en production.
Correction	Ajouter le fichier au dépôt lors d'une future opération Git volontaire.

20.12 Différence de casse entre Windows et Render/Linux

Exemple	Le fichier se nomme <code>Competences-v2.png</code> mais le code demande <code>competences-v2.png</code>
Symptôme	Le chemin peut sembler fonctionner sous Windows puis échouer sur un serveur sensible à la casse.
Diagnostic	Comparer caractère par caractère le chemin déployé.
Correction	Uniformiser exactement les noms et références.

20.13 classList utilisé sur null

Exemple	Un des éléments du tableau elementsAnimes n'existe pas
Symptôme	Erreur du type Cannot read properties of null, reading classList.
Diagnostic	Afficher chaque résultat de getElementById avant la boucle.
Correction	Corriger le HTML ou prévoir une validation explicite des éléments.

20.14 Préchargement pris pour une attente complète

Exemple	Supposer que les trois images sont chargées au retour de prechargerImages
Symptôme	Une image lente peut encore apparaître tardivement.
Diagnostic	Observer les requêtes dans l'onglet Réseau.
Correction	Écouter load/error ou utiliser des promesses si une attente devient nécessaire.

20.15 Clics rapides non gérés

Exemple	Cliquer sur Projets puis Compétences avant 300 ms
Symptôme	Clignotement, contenu intermédiaire ou classes retirées dans un ordre inattendu.
Diagnostic	Tracer le début de chaque clic et l'exécution de chaque callback.
Correction	Annuler les timers précédents, invalider les anciens callbacks ou verrouiller la transition.

21. Pistes d'amélioration futures

Cadre de cette section

Ces propositions ne sont pas des corrections déjà appliquées et ne signifient pas que le projet est inutilisable. Elles indiquent des étapes d'apprentissage possibles lorsque le besoin apparaîtra.

21.1 Remplacer les attributs onclick

Installer les écouteurs depuis JavaScript centraliserait les interactions et éviterait de dépendre de noms globaux dans le HTML.

21.2 Mettre les références DOM en cache

Les éléments statiques pourraient être recherchés une seule fois au chargement au lieu de refaire quatre recherches à chaque clic.

21.3 Écouter animationend

Le remplacement pourrait attendre la fin réelle du fade-out au lieu de dupliquer la valeur 300 entre JavaScript et CSS.

21.4 Gérer les clics rapides

Des identifiants de timer, clearTimeout, un numéro de transition ou un verrou simple pourraient invalider les anciens callbacks.

21.5 Mettre à jour aria-expanded

Le bouton pourrait annoncer son état ouvert ou fermé aux technologies d'assistance.

21.6 Fermer seulement sur un lien

L'écouteur du nav pourrait vérifier la cible afin qu'un clic dans une zone vide ne ferme pas le menu.

21.7 Respecter prefers-reduced-motion

Une media query pourrait réduire ou supprimer les animations pour les personnes sensibles aux mouvements.

21.8 Prévoir les erreurs d'image

Des écouteurs load et error permettraient un message ou une image de remplacement si un fichier manque.

21.9 Réduire l'usage de innerHTML

Si les contenus devenaient externes, une construction DOM sûre ou textContent limiterait les risques d'injection.

21.10 Découper lorsque le projet grandira

Les données, la navigation et les animations pourraient être séparées dans plusieurs modules uniquement lorsque la taille le justifiera.

22. Exercices de compréhension

Essayez de répondre sans consulter le chapitre suivant. Toutes les questions utilisent uniquement le code du portfolio. Une feuille séparée suffit pour noter vos réponses.

Méthode conseillée

Pour les chronologies, écrivez d'abord ce qui arrive immédiatement, puis ce qui arrive après au moins 300 ms. Pour les conditions, remplacez mentalement chaque propriété par true ou false.

Exercice 1 - Lire les objets

Reproduisez un tableau contenant, pour accueil, competences et projets, la valeur de titre, le chemin image et afficherProjets. Quel objet est le seul à demander l'affichage des cartes ?

Exercice 2 - Accéder aux propriétés

Donnez sans exécuter le code les valeurs de accueil.titre, competences.image, projets.afficherProjets, !projets.afficherProjets et !accueil.afficherProjets.

Exercice 3 - Comprendre sourcesImages

Quel est le contenu exact du tableau juste après sa création ? Si accueil.image était modifiée ensuite, la première valeur du tableau changerait-elle automatiquement ?

Exercice 4 - Comprendre const

Laquelle de ces opérations est interdite : réaffecter accueil avec un nouvel objet ou modifier accueil.titre ? Expliquez la différence entre liaison constante et objet immuable.

Exercice 5 - Paramètres et arguments

Dans changerContenu(projets, false), quels sont les arguments ? Quelles valeurs reçoivent contenu et premierAffichage ? L'objet projets est-il modifié ?

Exercice 6 - Étudier les conditions

Pour chacun des trois objets, donnez contenu.afficherProjets, sa négation, l'exécution ou non du bloc de masquage et l'exécution ou non du bloc d'affichage différé.

Exercice 7 - Retrouver l'ordre du démarrage

Remplacez dans l'ordre : création des objets, création de sourcesImages, préchargement, affichage Accueil, ajout de fade-in et installation des écouteurs du menu.

Exercice 8 - Premier affichage

Décrivez changerContenu(accueil, true). Le contenu attend-il 300 ms ? fade-out est-elle ajoutée ? Quel alt est produit ? Le bloc Projets est-il modifié ?

Exercice 9 - Afficher les projets

Décrivez changerContenu(projets, false) en séparant les opérations immédiates, celles exécutées après le délai et les durées des deux fade-in.

Exercice 10 - Quitter les projets

Supposez que visible est présente. Pourquoi ajouter cache-projets avant de retirer visible ? Combien de callbacks setTimeout sont programmés par changerContenu(competences, false) ?

Exercice 11 - Cas inhabituel

Que ferait `changerContenu(projets, true)` ? Le titre, le texte et l'image changeraient-ils ? Les cartes de projets seraient-elles affichées ? Justifiez avec les branches de la fonction.

Exercice 12 - Sélectionner le DOM

Listez les six identifiants recherchés par `getElementById`. Que renvoie la méthode si l'un d'eux n'existe pas ? Donnez deux lignes qui pourraient ensuite échouer.

Exercice 13 - Comprendre la closure

Pourquoi `mettreAJourContenu` peut-elle utiliser `contenu`, `titre`, `paragraphe` et `image` sans arguments ? Peut-on l'appeler directement depuis la portée globale ?

Exercice 14 - Parcourir les tableaux

Combien de fois le callback de préchargement est-il exécuté et quelles valeurs reçoit `sourceImage` ? Quelles références reçoit `element` dans `changerContenu` ?

Exercice 15 - Comparer les callbacks

Relevez un callback de `forEach`, un callback de `setTimeout` et un callback de `addEventListener`. Expliquez pourquoi ils n'ont pas le même moment d'exécution.

Exercice 16 - Préchargement réel

Vrai ou faux : les objets `Image` sont insérés dans la page ; affecter `src` peut démarrer le chargement ; la fonction attend toutes les images ; une erreur est gérée ; le cache peut être réutilisé.

Exercice 17 - Mettre à jour Compétences

Donnez les valeurs affectées à `titre.innerHTML`, `paragraphe.innerHTML`, `image.src` et `image.alt`. Pourquoi `innerHTML` est-il utilisé pour le paragraphe ?

Exercice 18 - Manipuler les classes

Expliquez `add`, `remove` et `toggle`. Quel est le résultat de deux clics successifs sur le burger sans autre interaction entre les deux ?

Exercice 19 - Classer les notions

Classez `changerContenu`, `setTimeout`, `forEach`, `getElementById`, `addEventListener`, `Image`, `innerHTML`, `src`, `new` et `!` en fonction, méthode, constructeur, propriété ou opérateur.

Exercice 20 - Diagnostiquer les clics rapides

Pourquoi un clic sur Projets suivi de Compétences avant 300 ms laisse-t-il plusieurs callbacks actifs ? Quels symptômes sont possibles et quelle stratégie future pourrait les éviter ?

23. Corrections expliquées

Correction 1 - Lire les objets

Accueil contient Bienvenue sur mon portfolio, images/villedenuit1.jpg et false. Compétences contient Mes compétences, images/competences-v2.png et false. Projets contient Mes projets, images/liberté.jpg et true. Projets est le seul objet qui demande les cartes.

Correction 2 - Accéder aux propriétés

Les valeurs sont, dans l'ordre : Bienvenue sur mon portfolio ; images/competences-v2.png ; true ; false ; true. L'opérateur ! inverse le résultat sans modifier les propriétés.

Correction 3 - Comprendre sourcesImages

Le tableau contient images/villedenuit1.jpg, images/competences-v2.png et images/liberté.jpg. Une modification ultérieure de accueil.image ne changerait pas automatiquement sa première entrée : les chaînes primitives ont été copiées lors de la création.

Correction 4 - Comprendre const

Réaffecter accueil à un nouvel objet est interdit. Modifier accueil.titre reste techniquement possible, car const protège la liaison et ne gèle pas les propriétés. Le projet actuel ne réalise aucune mutation.

Correction 5 - Paramètres et arguments

Les arguments sont projets et false. contenu référence projets et premierAffichage reçoit false. contenu.afficherProjets vaut donc true. La fonction lit l'objet sans le modifier.

Correction 6 - Étudier les conditions

Pour accueil et competences : false, !false vaut true, le masquage s'exécute et l'affichage différé ne s'exécute pas. Pour projets : true, !true vaut false, le masquage est ignoré et visible est ajoutée après le délai. Ces réponses supposent premierAffichage égal à false.

Correction 7 - Retrouver l'ordre du démarrage

L'ordre est : trois objets, sourcesImages, appel de prechargerImages et ses trois itérations, appel de changerContenu(accueil, true), mise à jour et fade-in, puis recherche du burger et installation des écouteurs. Une déclaration de fonction ne lance pas son corps ; l'appel le fait.

Correction 8 - Premier affichage

Le contenu est affecté immédiatement. Aucune fade-out et aucun setTimeout de changerContenu ne sont créés. alt devient Illustration de Bienvenue sur mon portfolio. Le bloc Projets est sélectionné mais ses classes ne sont pas modifiées ; son display: none de base le cache.

Correction 9 - Afficher les projets

Immédiatement, les trois éléments passent de fade-in à fade-out. Après au moins 300 ms, les données Projets sont affectées, visible est ajoutée et les trois éléments repassent en fade-in. Les cartes entrent en 0,3 seconde ; le contenu principal entre en 2 secondes.

Correction 10 - Quitter les projets

visible doit rester présente pendant la sortie afin de conserver display: flex. cache-projets lance fadeOut. Deux callbacks sont programmés : le premier nettoie visible et cache-projets, le second remplace le contenu et relance fade-in.

Correction 11 - Cas inhabituel

Le titre, le texte, src et alt seraient mis à jour immédiatement et fade-in serait ajoutée. Les cartes ne recevraient pas visible, car cette instruction se trouve seulement dans la branche else. Ce cas montre que true est prévu pour l'initialisation d'Accueil.

Correction 12 - Sélectionner le DOM

Les identifiants sont titre2, textep, monImage, projetsLiens, menuBurger et menuNavigation. Une recherche sans résultat renvoie null. titre.innerHTML, element.classList, boutonBurger.addEventListener ou navigationMobile.classList pourraient alors déclencher une erreur.

Correction 13 - Comprendre la closure

La fonction interne conserve l'accès à l'environnement lexical de changerContenu. Elle retrouve donc le paramètre et les constantes locales. Son nom n'existe pas hors de cette fonction et ne peut pas être appelé directement depuis la portée globale.

Correction 14 - Parcourir les tableaux

Le callback de préchargement s'exécute trois fois avec les trois chemins dans l'ordre. Dans elementsAnimes, element reçoit le h2, le paragraphe puis l'image. forEach évite de répéter la même manipulation de classe trois fois.

Correction 15 - Comparer les callbacks

forEach appelle son callback immédiatement pendant la boucle. setTimeout le rend exécutable après un délai minimal. addEventListener le conserve jusqu'à l'événement click. La syntaxe fléchée peut être identique, mais l'organisme receveur décide du moment.

Correction 16 - Préchargement réel

Réponses : faux, vrai, faux, faux, vrai. Les objets Image restent en mémoire ; src peut lancer une requête ; aucune attente load ni gestion error n'existe ; le navigateur peut réutiliser son cache.

Correction 17 - Mettre à jour Compétences

titre.innerHTML reçoit Mes compétences ; paragraphe.innerHTML reçoit competences.texte ; image.src reçoit images/competences-v2.png ; image.alt reçoit Illustration de Mes compétences. innerHTML est utilisé afin d'interpréter la balise br du texte contrôlé.

Correction 18 - Manipuler les classes

add ajoute, remove retire et toggle bascule selon la présence. Au premier clic, ouvert est ajoutée et la navigation responsive apparaît. Au second clic, ouvert est retirée et elle disparaît. Sur ordinateur, le burger est masqué par le CSS.

Correction 19 - Classer les notions

changerContenu et setTimeout sont des fonctions ; forEach, getElementById et addEventListener sont des méthodes ; Image est un constructeur ; innerHTML et src sont des propriétés ; new et ! sont des opérateurs. src peut être une propriété de plusieurs objets image différents.

Correction 20 - Diagnostiquer les clics rapides

Chaque appel programme ses propres callbacks et le code ne conserve pas leurs identifiants. Un ancien callback peut remplacer temporairement le dernier choix ou modifier les classes au mauvais moment. Une évolution pourrait annuler les timers avec clearTimeout, numéroté les transitions ou utiliser animationend avec une validation du dernier choix.

24. Glossaire final

Les définitions restent volontairement simples et utilisent le vocabulaire du projet.

Terme	Définition liée au projet
Affectation	Action de donner une valeur, comme <code>image.src = contenu.image</code> .
Argument	Valeur fournie lors d'un appel, comme <code>projets</code> dans <code>changerContenu(projets, false)</code> .
Asynchrone	Travail qui reprend plus tard sans bloquer toute l'interface, comme un callback de <code>setTimeout</code> .
Attribut HTML	Information d'une balise ; <code>src</code> , <code>alt</code> , <code>id</code> et <code>class</code> en sont des exemples.
Booléen	Valeur logique <code>true</code> ou <code>false</code> .
Boucle d'événements	Mécanisme qui permet au navigateur d'exécuter les callbacks lorsqu'ils sont prêts.
Cache	Zone où le navigateur peut conserver une ressource déjà demandée.
Callback	Fonction transmise à un autre mécanisme pour être appelée au moment approprié.
Chaîne de caractères	Valeur textuelle, comme <code>Mes projets</code> ou un chemin d'image.
Classe CSS	Nom reliant un élément à des règles, comme <code>fade-in</code> , <code>visible</code> ou <code>ouvert</code> .
<code>classList</code>	Objet permettant de consulter et modifier les classes d'un élément DOM.
Closure	Fonction qui conserve l'accès à son environnement lexical.
Condition	Structure qui choisit un chemin, comme <code>if</code> (<code>premierAffichage</code>).
Constante	Liaison déclarée avec <code>const</code> et impossible à réaffecter.
Constructeur	Fonction destinée à créer des objets avec <code>new</code> , comme <code>Image</code> .
DOM	Représentation JavaScript de la page HTML.
<code>DOMTokenList</code>	Type d'objet représenté ici par <code>classList</code> , spécialisé dans une liste de classes.
Élément DOM	Objet JavaScript représentant une balise réelle de la page.
Événement	Signal produit par le navigateur, comme un clic.
Fermeture lexicale	Expression française pour closure.
Fonction	Bloc de code réutilisable, comme <code>prechargerImages</code> .
Fonction fléchée	Syntaxe utilisant <code>=></code> , employée ici pour les callbacks.
Fonction interne	Fonction déclarée dans une autre fonction, comme <code>mettreAJourContenu</code> .
<code>forEach</code>	Méthode de tableau qui appelle un callback une fois par valeur.
<code>getElementById</code>	Méthode de document qui recherche un élément par son identifiant.
Hoisting	Mécanisme rendant une déclaration de fonction disponible avant son appel dans le même script.
Identifiant	Valeur HTML unique utilisée pour retrouver un élément, comme <code>titreh2</code> .
<code>if / else</code>	Structure qui sélectionne une branche selon le résultat d'une condition.
<code>Image</code>	Constructeur fourni par le navigateur pour créer un objet image.
<code>innerHTML</code>	Propriété permettant de lire ou remplacer le contenu HTML interne.
Interpolation	Insertion d'une expression dans une chaîne avec <code>\${...}</code> .

Terme	Définition liée au projet
Itération	Un passage d'une boucle ; le préchargement en compte trois.
Liaison	Association entre un nom de variable et une valeur.
Méthode	Fonction rattachée à un objet, comme classList.add.
Milliseconde	Un millième de seconde ; 300 ms représentent 0,3 seconde.
Mutation	Modification d'un objet existant, par exemple changer une propriété.
Négation logique	Opérateur ! qui inverse temporairement une valeur logique.
new	Opérateur demandant la création d'une instance à partir d'un constructeur.
null	Valeur représentant ici l'absence d'élément trouvé dans le DOM.
Objet	Ensemble de propriétés, comme accueil.
Objet littéral	Objet écrit directement avec des accolades.
Paramètre	Nom local qui reçoit un argument, comme contenu.
Préchargement	Déclenchement anticipé du chargement d'une ressource.
Propriété	Valeur rattachée à un objet, comme projets.image.
Propagation	Parcours d'un événement dans les éléments parents du DOM.
Portée	Zone du code dans laquelle un nom est accessible.
Portée globale	Zone extérieure aux fonctions dans le script.
Portée locale	Zone limitée à une fonction, un callback ou un bloc.
Réaffectation	Remplacement complet de la valeur liée à une variable.
Référence	Lien par lequel une variable désigne un objet existant.
setTimeout	Fonction qui programme un callback après un délai minimal.
src	Propriété ou attribut indiquant la ressource d'une image.
Synchrone	Exécuté immédiatement dans le flux courant, comme forEach dans ce projet.
Tableau	Collection ordonnée, comme sourcesImages ou elementsAnimes.
Template literal	Chaîne entre backticks pouvant être multiligne et interpolée.
Temporisation	Programmation d'un travail différé.
toggle	Méthode qui ajoute une classe absente ou retire une classe présente.
Valeur	Donnée associée à une propriété, un argument ou une variable.
Valeur primitive	Valeur simple comme une chaîne ou un booléen, copiée par valeur.
Variable	Nom permettant de retrouver une valeur dans une certaine portée.
undefined	Valeur obtenue notamment lorsqu'un argument ou une propriété manque.
Écouteur d'événement	Fonction enregistrée avec addEventListener pour réagir à un événement.

Dernier repère

Pour relire le script, posez toujours quatre questions : quelle valeur circule ? quelle condition est évaluée ? quel élément DOM est modifié ? quelle règle CSS produit l'effet visible ?